



CGNS Proposal for Extension 0050

DOF Storage Type

Version 1

July 29, 2026

CPEX#	0050
Scope	Record DOF storage layout: shared or independent
Champion	M. Scot Breitenfeld
Organization	The HDF Group
E-mail	brtnfld@hdfgroup.org
GitHub Issue	to be assigned by the CPEX editors
Reference Impl.	pending (see Section 12)
Target Release	pending (see Section 12)
Date First Posted	April 21, 2026
Date of Last Revision	July 29, 2026
Prior Registry Entry	Listed while in draft as “Support Solution Continuity Type in CGNS” (CPEX-0050-SolutionContinuity.pdf); see Section 2.6
SIDS Status	proposal under review
Filemap Status	proposal under review
MLL Status	proposal under review

Contents

1 Abstract	4	9
2 Motivation and Rationale	4	10
2.1 Problem Statement	4	11
2.2 Evidence from Practice: Grid Duplication as a Workaround	5	12
2.3 Inadequacy of Existing CGNS Constructs	6	13

2.4	Scope and Non-Goals	7	14
2.5	Prior Art: The DOF-Map Concept in ASCI-DMF / SAF	8	15
2.6	Naming History	9	16
3	Detailed Description	9	17
3.1	Design Overview	9	18
3.2	New Enumeration: <code>DofStorage_t</code>	9	19
3.2.1	Caveat: Vector-Conforming Spaces	10	20
3.2.2	Rationale for Two-Value Classification	11	21
3.2.3	Enumeration Value Naming	12	22
3.3	SIDS Node Definition	13	23
3.3.1	Node Placement Rationale	15	24
3.3.2	Why a New Node Rather Than a New <code>GridLocation_t</code> Value	15	25
3.4	New MLL Functions	16	26
3.4.1	Design: Position-Based API	16	27
3.4.2	C API	16	28
3.4.3	Fortran API	17	29
3.5	Internal Data Structure Changes	18	30
3.6	Internal Lookup Table and String Mapping	18	31
3.7	MLL Implementation Requirements	19	32
3.7.1	Address Resolution Function (<code>cgns_internals.c</code>)	19	33
3.7.2	Internal Tree Parser Update (<code>cgns_internals.c</code>)	20	34
3.7.3	Read Function Implementation	20	35
3.7.4	Write Function Implementation	21	36
3.8	Data Layout for Independent DOFs at Shared Mesh Vertices	22	37
3.8.1	Applicability and Restrictions	22	38
3.8.2	Other Grid Locations	24	39
3.8.3	DOF Ordering Rule	24	40
3.9	Interaction with <code>Rind_t</code> (Ghost Layers)	25	41
4	Interaction with CPEX 0045 (Higher-Order Elements)	26	42
4.1	Recommended Practice	26	43
4.2	<code>DofStorage_t</code> Under <code>InterpolationPoints</code>	27	44
5	Interaction with CPEX 0047 (Integration Points)	27	45
6	Backward Compatibility	28	46
6.1	Parallel I/O	29	47
6.2	Interpreting Legacy Data (<code>DofStorageNull</code>)	29	48
7	Examples	30	49
7.1	Example 1: Writing a Shared-DOF (CG) Solution	30	50
7.2	Example 2: Writing an Independent-DOF (DG) Solution	30	51

7.3	Example 3: Reading and Dispatching on DOF Storage Layout	31	52
7.4	Example 4: HDG Solver with Both DOF Storage Types	32	53
7.5	Example 5: Fortran Usage	33	54
7.6	Example 6: Low-Order Finite Volume (DG $p = 0$)	33	55
7.7	Example 7: Independent DOFs on a Shared-Vertex Mesh	33	56
7.8	Example 8: <code>DiscreteData_t</code> with an Independent Error Indicator .	35	57
7.9	Example 9: <code>ZoneSubRegion_t</code> with Boundary Flux Data	35	58
7.10	Example 10: Solver Restart with DOF-Storage-Aware Read	35	59
7.11	Example 11: FSI Coupling — Structural and Fluid Solutions	36	60
7.12	Example 12: Conjugate Heat Transfer — Perfect and Imperfect Thermal Contact	37	62
8	SIDS Impact	38	63
9	File Mapping Impact	39	64
9.1	Affected File Mapping Figures	39	65
9.2	Writing Conventions	40	66
10	Design Decisions	40	67
10.1	Rejected Alternatives	41	68
11	Recommended <code>cgnscheck</code> Validation Rules	42	69
12	Reference Implementation, Versioning, and Test Plan	43	70
12.1	Deliverables	43	71
12.2	Versioning	44	72
12.3	Test Plan	44	73
13	Summary of API Additions	45	74
14	References	46	75

1 Abstract

This CPEX introduces a `DofStorage_t` enumeration and corresponding SIDS node to the CGNS standard, enabling CGNS files to explicitly record how degrees of freedom (DOFs) are stored relative to shared mesh entities. The two values—**Shared** and **Independent**—answer a single question: for DOFs located at a geometric entity (vertex, edge, or face) that is shared between adjacent elements, does each adjacent element index the *same* buffer slot (shared storage), or does each element own its *own* independent slot (independent storage)? The question is always posed relative to a stated reference entity set—for a block indexed over volume elements those are the volume elements; for an interface block they are the volume elements adjacent to each listed face or edge (Section 4.2). This is a statement about data layout only; it makes no claim about the mathematical smoothness of the resulting field.

The gap being closed is specific. For every encoding legal today, the DOF layout is already fixed by `GridLocation_t` together with the SIDS `DataSize` rules, so this proposal is not a remedy for ambiguity. What cannot presently be expressed at all is an element-local layout at a shared-entity grid location: a solver storing per-element solution values at positions that coincide with shared mesh vertices has no conforming encoding, and the workaround observed in practice is to duplicate the grid coordinates, destroying the shared-vertex connectivity that mesh walking depends on (Section 2.2). Accordingly **Independent** at `GridLocation_t = Vertex`, together with the `PointList` ordering rule of Section 3.8, is the load-bearing content of this proposal; **Shared** is retained for explicit self-description of the layout that the connectivity already implies. The node is defined for any polynomial order and any discretization family, and complements—without duplicating—the higher-order solution infrastructure of CPEX 0045 [4] and the integration-point infrastructure of CPEX 0047 [5], both of which are element-local by construction and require no such designator.

2 Motivation and Rationale

2.1 Problem Statement

Modern CFD and computational mechanics codes increasingly use discretization methods whose solution data cannot be fully described by the existing CGNS metadata [1]. The core issue is the treatment of degrees of freedom (DOFs) at shared geometric entities (vertices, edges, faces):

- In a **shared** layout (e.g., Continuous Galerkin FEM, C^0 spectral element methods), DOFs at shared interfaces are *shared*: a single value exists per geometric node, and the global solution field is at least C^0 -continuous across element boundaries.

- In an **independent** layout (e.g., Discontinuous Galerkin [6, 7], finite volume, C^{-1} spectral element methods), DOFs are *element-local*: each element owns its own values at every point, including at shared interfaces. Multiple independent values may exist at the same geometric location.

This distinction affects:

1. **Data size and layout.** A DG solution on a mesh with shared vertices stores more values than a CG solution on the same mesh. Because the SIDS sizing rules for **Vertex** admit only the shared count, such a solution cannot be written at that grid location today; once a second sizing is admitted, a machine-readable discriminator is required so that readers can tell which of the two is in use.
2. **Visualization and interpolation.** Shared-DOF fields can be interpolated globally using the mesh connectivity; independent-DOF fields require element-local interpolation and may exhibit intentional discontinuities at element interfaces that should not be smoothed.
3. **Restart and data exchange.** Solvers reading restart files must know whether DOFs are shared or element-local to correctly reconstruct the solution state. Coupling frameworks exchanging solution data between codes need this metadata to perform correct inter-code transfers. In Fluid-Structure Interaction (FSI) applications, for example, a continuous FEM structural displacement field and a discontinuous DG fluid field coexist—the coupling framework must use different interpolation and projection operators for each, and the DOF storage metadata enables this dispatch.
4. **Error estimation and adaptation.** The jump in solution values across element interfaces in a DG solution is physically meaningful and is used for error estimation. A post-processor that incorrectly treats DG data as shared-DOF data would average out these jumps.

2.2 Evidence from Practice: Grid Duplication as a Workaround

The requirement addressed here was raised by implementors of third-party CGNS readers, whose difficulty is concrete rather than theoretical.¹ The substance of their report, paraphrased:

At least one DG solver in production writes *duplicate grid coordinates* for nodes that are geometrically shared, so that the grid and the solution have matching extents and existing readers will accept the file. What is needed is a designator for the case of shared nodes in the grid but

¹Raised in correspondence with the CPEX champion, April 2026. Specific parties and codes are intentionally not identified.

independent solution values at those nodes. Retaining shared grid nodes
is what permits walking through the mesh—stream tracing, for example.

This is the expressiveness gap of Section 2.5 observed in the field, and it is the
primary justification for this proposal. Because no designator exists, a solver wishing
to emit element-local solution values at vertex positions must instead *explode the*
mesh: every element receives its own copy of each vertex, the `GridCoordinates_t`
array grows by the average vertex valence, and the shared-vertex connectivity that
makes mesh walking possible is destroyed. The cost is borne twice—once in file size,
and again by every downstream reader that can no longer traverse the mesh.

Two points about this evidence deserve emphasis, because they determine how narrow
the proposal should be:

- The requirement is specifically for *independent* DOFs at `GridLocation_t =`
`Vertex` while the grid retains shared vertices. It is a request for one designator
value, not for a general classification of DOF storage.
- The constituency is third-party *readers*, not solver authors. This matters
when weighing this proposal against the alternative of requiring CPEX 0045's
`InterpolationPoints`: a reader that must implement `Family_t`-level
`SolutionInterpolation_t` machinery in order to interpret a first-order DG
field bears a cost out of proportion to the data, and CPEX 0045 is itself not yet
part of the standard.

2.3 Inadequacy of Existing CGNS Constructs

No existing CGNS mechanism captures the shared/independent distinction:

`GridLocation_t` specifies *where* data is located (vertices, cell centers, faces, edges,
interpolation points, or integration points), but not whether values at shared
locations are shared or element-local. A `Vertex`-located solution could use either
layout; the same applies to `InterpolationPoints` and `IntegrationPoint`.

`PointSet` (`PointList`, `PointRange`) can index subsets of data but does not encode
DOF-sharing semantics.

`InterpolationType_t` (CPEX 0045 [4]) describes the polynomial basis (e.g., La-
grange, monomial), but not whether that basis yields a globally shared or
element-local DOF set.

`SolutionInterpolation_t` (CPEX 0045, a child of `Family_t`) records the inter-
polation type and control-point layout for a given basic element type; the
polynomial orders are recorded separately in the `InterpolationOrders` child
of `FlowSolution_t`. Neither carries DOF-sharing information.

`ItgPointsStartOffset` (CPEX 0047 [5]) gives the per-element starting offsets of

integration-point data, which implies an element-local layout for that particular
 grid location, but says nothing about vertex-, edge-, or face-located DOFs and
 does not generalize to other grid locations.

Data array size together with `GridLocation_t` *does* determine the layout for
 every encoding legal before this CPEX, and it should be stated plainly that
 this is so: the `SIDS DataSize` formula admits exactly one layout at each grid
 location (Section 6.2 tabulates them), and it does so irrespective of element
 mix. What the sizing rules cannot do is admit an element-local layout at a
 shared-entity grid location at all—that layout is not merely ambiguous, it is
 inexpressible. The gap this CPEX addresses is therefore one of expressiveness,
 not of ambiguity; see Section 2.5.

2.4 Scope and Non-Goals

This proposal is deliberately narrow. Stating plainly what it does and does not close
 is the clearest way to show that it neither overlaps CPEX 0045 nor CPEX 0047.

What it closes. One gap: element-local solution DOFs at positions coinciding
 with shared mesh vertices, with the shared-vertex grid retained. This is the case
 of Section 2.2, it has no conforming encoding today, and it is not restricted to
 high-order methods—DG $p = 1$ on a linear mesh is the most common instance, and
 the predefined element types extend the same pattern through fourth order. Because
 the affected node is `FlowSolution_t` (and its `DiscreteData_t`, `ZoneSubRegion_t`,
 and `UserDefinedData_t` counterparts), the designator belongs there rather than
 inside the higher-order interpolation infrastructure.

What it does not close, by design.

- **Nothing that is already determined.** At `CellCenter`, under CPEX 0045’s
`InterpolationPoints`, and under CPEX 0047’s `IntegrationPoint`, the layout
 follows from the grid location and the sizing rules. Recording it there is permitted,
 and recommended for uniformity, but it resolves nothing and this proposal does
 not claim otherwise.
- **Higher-order DOF identification.** Establishing which DOFs of two adjacent
 elements correspond across a shared face—the general dof-relation—requires
 per-element index mapping and orientation conventions that this proposal does
 not supply. For arbitrary interpolation points it cannot be done at all from the
 metadata CGNS currently defines (Section 4.2).
- **Field continuity.** Whether duplicated DOF values agree by construction—
 the difference between a continuous field stored redundantly and a genuinely

discontinuous one—is a distinct property, not derivable from storage layout, and outside the scope of this node.

- **Structured zones**, `NFACE_n` sections, and vector-conforming space orientation (Sections 3.8.1 and 3.2.1).

2.5 Prior Art: The DOF-Map Concept in ASCI-DMF / SAF

The concept encoded by `DofStorage_t` has well-established prior art in the scientific data management literature. The ASCI Data Model Framework (ASCI-DMF) and its implementation in the Sets and Fields (SAF) library [8] introduced the *dof-relation*: given a flat buffer of DOF values, the *dof-relation* enumerates for each element which buffer indices are assigned to it. When two adjacent elements refer to the *same* buffer index for a shared interface entity, that DOF is shared—this is `DofStorage_t = Shared` in the present proposal. When each element refers to *different* buffer indices (each element owns its own slot), the DOFs are independent—corresponding to `DofStorage_t = Independent`.

Miller [8] notes that for piecewise-linear coordinate fields the *dof-relation* and the mesh connectivity (*topo-relation*) are one and the same array, and are consequently “hopelessly confused” in practice. CGNS today is in exactly this position: it has no *dof-relation* layer, so the only *dof-map* available to a solution at a shared-entity grid location is the one implied by the `Elements_t` connectivity. This does not make existing files ambiguous—a conforming file is unambiguous, because `GridLocation_t` together with the SIDS `DataSize` formula fixes the layout in every legal case (Section 6.2 tabulates them). The cost is expressiveness: because the implied *dof-map* is the connectivity, a solution whose DOFs are element-local cannot be written at `Vertex`, `EdgeCenter`, or `FaceCenter` at all. CPEX 0045 supplies an independent per-element *dof-map* for `InterpolationPoints`, and CPEX 0047 does so for `IntegrationPoint`; both are element-local by construction. What remains uncovered is element-local DOFs at `Vertex` on the predefined element types, where `DofStorage_t` serves as the discriminator that makes a second sizing both legal and self-describing.

It is important to note that `DofStorage_t` encodes only the *coarsest* level of DOF-map metadata—shared versus independent—which is sufficient for the vast majority of post-processing and data-exchange use cases. The full generality of a *dof-relation* (arbitrary per-element index remapping, as required for quadratic or higher-order elements whose mid-edge DOFs must be identified across elements) is the domain of CPEX 0045 [4] and CPEX 0047 [5]. These CPEXs are therefore complementary layers of the same concept introduced in ASCI-DMF.

2.6 Naming History

This proposal has not yet been presented, so the document carries no revision history. It did, however, pass through two working names before this one, and both left traces that a reader may encounter—hence this note.

An early draft used `SolutionRepresentation_t`, which is where the former source filename `CPEX-0050-solution-representation.tex` came from. A later draft used `SolutionContinuity_t`, with values `Continuous` and `Discontinuous`, and it is under that name that the CPEX registry currently lists the entry (scope line “Support Solution Continuity Type in CGNS,” attachment `CPEX-0050-SolutionContinuity.pdf`). That vocabulary proved misleading: readers consistently understood “continuity” as a claim about the mathematical smoothness of the field, which this node does not make.

The name adopted here is `DofStorage_t`, with values `Shared` and `Independent`, chosen to state plainly that the node describes *storage layout*. The source document and its PDF are named to match (`CPEX-0050-dof-storage`). **The registry entry requires a corresponding update** to its scope line and attachment. No files have been written with either earlier name—there is no reference implementation yet—so no migration path is required.

3 Detailed Description

3.1 Design Overview

The proposal introduces:

1. A new `DofStorage_t` enumeration with values covering the common DOF storage layouts (Section 3.2).
2. A new optional child node, `DofStorage`, of `FlowSolution_t`, `DiscreteData_t`, `ZoneSubRegion_t`, and `UserDefinedData_t`, recording the DOF storage layout (Section 3.3).
3. One pair of position-based Mid-Level Library [3] functions to read and write the DOF storage layout, following the pattern established by `cg_gridlocation_read` / `cg_gridlocation_write` (Sections 3.4 and 3.7).
4. Fortran bindings for that pair (Example 7.5).

3.2 New Enumeration: `DofStorage_t`

```

1 typedef enum {
2     CGNS_ENUMV( DofStorageNull )           = CG_Null,
3     CGNS_ENUMV( DofStorageUserDefined )    = CG_UserDefined,
4     CGNS_ENUMV( Shared )                   = 2,
5     CGNS_ENUMV( Independent )              = 3
6 } CGNS_ENUMT( DofStorage_t );
7
8 #define NofValidDofStorage 4
9
10 extern CGNSDLL const char *DofStorageName[NofValidDofStorage];

```

The enumeration values are:

DofStorageNull No DOF storage layout specified. This is the default for backward compatibility: existing files and codes that do not set this field are unaffected. See Section 6.2 for the recommended interpretation of legacy data.

DofStorageUserDefined Application-defined DOF storage layout not covered by the standard values. When this value is used, writers *shall* attach a **Descriptor_t** child node named "DofStorageDescription" to the same parent node. The string value identifies the storage convention (e.g., "HDG-trace", "C1-spline"). This allows readers to recognize the convention without requiring a separate side-channel; see Example 7.3 for the reader side.

Shared DOFs at shared mesh entities (vertices, edges, faces) are *shared* between adjacent elements: each shared entity carries exactly one value. This is a statement about *storage and DOF-sharing semantics*, not about the mathematical smoothness of the field. Typical of: Continuous Galerkin FEM, C^0 spectral element methods, isogeometric analysis with C^0 junctions. See the caveat on vector-conforming spaces below.

Independent The solution field is element-local: each element owns independent DOFs, including at shared geometric locations, and no interface DOF is shared between adjacent elements. Typical of: Discontinuous Galerkin methods, finite volume methods, C^{-1} spectral element methods.

3.2.1 Caveat: Vector-Conforming Spaces

$\mathbf{H}(\text{div})$ -conforming spaces (e.g., Raviart–Thomas) and $\mathbf{H}(\text{curl})$ -conforming spaces (e.g., Nédélec) store one value per shared interface entity—a normal-flux or tangential-component DOF, respectively—and so satisfy the letter of the **Shared** definition, even though the full vector field is not pointwise C^0 -continuous.

However, such DOFs additionally carry a per-element **orientation convention**: the sign of the stored value depends on a choice of outward normal ($\mathbf{H}(\text{div})$) or edge tangent direction ($\mathbf{H}(\text{curl})$), and adjacent elements see the same DOF with

opposite sign. `DofStorage_t` does not encode that orientation, and no existing CGNS construct does either.

Accordingly:

- A writer *may* label such a field **Shared**, since the storage statement is accurate.
- A writer that does so *shall* also mark the field with `DofStorageUserDefined`-style provenance—either by using `DofStorageUserDefined` with a `DofStorageDescription` of "Hdiv-RT" or "Hcurl-Nedelec", or by attaching a `Descriptor_t` identifying the space—so that readers are not misled.
- A reader *shall not* interpolate, average, or otherwise treat **Shared** DOFs as pointwise nodal values of a scalar field without first establishing that the field is nodal. Applying nodal interpolation to Raviart–Thomas or Nédélec DOFs yields incorrect results.

Supplying the orientation metadata that would make vector-conforming spaces safely self-describing is deliberately out of scope for this CPEX and is a natural subject for a future, narrowly scoped proposal.

3.2.2 Rationale for Two-Value Classification

The two values are not of equal weight, and the proposal is clearer for saying so.

Independent is load-bearing: at `GridLocation_t = Vertex` it admits a layout that has no other conforming encoding, and a reader must consult it to size and interpret the field correctly (Section 3.8).

Shared is confirmatory. Wherever it is valid, the sharing it describes is already realized by the `Elements_t` connectivity, so a reader that ignores the node reaches the same conclusion. It is retained for three reasons: it makes the layout explicit rather than implied, so files are self-describing; it gives writers a way to state the layout positively instead of relying on the absence of **Independent**; and an enumeration offering only one substantive value would be an odd construct with no precedent in the SIDS. It is *not* retained on the basis that readers require it, and Section 2.4 should be read as limiting the claims made for it. In particular, **Shared** is meaningless at $p = 0$ and invalid on a volume `InterpolationPoints` block (Section 4.2).

With that asymmetry stated, methods that appear to require finer distinctions are handled by the two values combined with standard CGNS patterns:

HDG and mixed-layout methods use both element-local volume DOFs and face-local trace DOFs. Note that the trace space of the standard hybridizable formulation is *not* globally continuous: it consists of face-wise polynomials with no continuity imposed between adjacent faces, so trace DOFs are generally discontinuous along skeleton edges and vertices. **Shared** on a trace block

therefore asserts only that the two volume elements adjacent to a given face index the same DOF block for that face—the reference entity set is the adjacent *volume* elements, per Section 4.2—and asserts nothing about continuity across the skeleton. The two layouts are stored as separate `FlowSolution_t` nodes, following the same pattern CGNS already uses to separate fields at different `GridLocation_t` values. A dedicated `MixedStorage` value would conflate two distinct data layouts into a single node, making it harder, not easier, for post-processors to interpret the data.

Penalized / weakly-continuous DG methods enforce continuity through penalty terms in the variational formulation, but the data layout remains element-local: each element owns its own DOFs. The correct label is **Independent**. “Weak continuity” is a solver-internal property of the numerical method, not a property of the stored data that readers and post-processors need to act on.

Isogeometric analysis (C^k , $k \geq 1$) uses basis functions with higher-than- C^0 smoothness, but the data layout still shares DOFs at patch interfaces. The correct label is **Shared**. The smoothness order k is a property of the basis, not of the data layout; it belongs in the interpolation metadata (CPEX 0045), not in the DOF storage type.

The `DofStorageUserDefined` value remains available as an escape hatch, consistent with CGNS convention for all enumeration types.

Scope note. `DofStorage_t` captures *storage and DOF-sharing semantics* only—it answers “are interface DOFs shared or element-local?” It does not encode the polynomial basis, smoothness order, patch topology, orientation conventions (Section 3.2.1), or support-radius metadata that isogeometric analysis (IGA), meshless, or other non-standard discretizations may additionally require. Those richer descriptions belong in CPEX 0045, CPEX 0047, or in future extensions; this CPEX is intentionally scoped to the coarser, universally applicable shared/independent distinction.

3.2.3 Enumeration Value Naming

The values **Shared** and **Independent** are unprefixes, following CGNS convention: only the **Null** and **UserDefined** members of an enumeration carry the type-name prefix (cf. `GridLocationNull` versus `Vertex`, `MassUnitsNull` versus `Kilogram`).

The committee should nonetheless weigh a known risk. Existing unprefixes CGNS enumeration values are domain-specific nouns (**Vertex**, **CellCenter**, **Kilogram**, **Structured**), whereas **Shared** and especially **Independent** are generic adjectives that applications and I/O layers commonly define for unrelated purposes. Because `cgnslib.h` injects enumeration values into the global C namespace, a collision would surface as a compile error in downstream code. No collision exists in `cgnslib.h`

today.

The alternative—`DofStorageShared` and `DofStorageIndependent`—trades a small departure from convention for collision safety. This proposal retains the unprefix names for consistency; the champion will adopt the prefixed names if the committee prefers. Adopting them would change only the C and Fortran identifiers, not the on-disk strings, so it is a source-compatibility question rather than a file-format one.

3.3 SIDS Node Definition

The DOF storage layout is stored as a new optional child node of `FlowSolution_t`. The amended SIDS definition adds one line (shown with the + marker). The definition below also shows the `InterpolationOrders` and `CharacteristicLength` children introduced by CPEX 0045 [4], for context; note that CPEX 0045's `SolutionInterpolation_t` node is a child of `Family_t`, not of `FlowSolution_t`.

`FlowSolution_t` (amended, SIDS Section 7.7)

```
FlowSolution_t< int CellDimension, int IndexDimension,
int VertexSize[IndexDimension],
int CellSize[IndexDimension] > :=
{
List( Descriptor_t Descriptor1 ... DescriptorN ) ;           (o)
GridLocation_t GridLocation ;                               (o/d)
IndexArray_t<1, 2, int> InterpolationOrders ;               (o/d)
(CPEX 0045)
dataArray_t<R8, 1|2, ...> CharacteristicLength ;           (o)
(CPEX 0045)
Rind_t<IndexDimension> Rind ;                               (o/d)
IndexRange_t<IndexDimension> PointRange ;                  (o)
IndexArray_t<IndexDimension, ListLength[], int> PointList ; (o)
List( dataArray_t<DataType, IndexDimension, DataSize[]>
dataArray1 ... dataArrayN ) ;                               (o)
DataClass_t DataClass ;                                     (o)
DimensionalUnits_t DimensionalUnits ;                       (o)
+   DofStorage_t DofStorage ;                               (o)
List( UserDefinedData_t UserDefinedData1 ...
```

```
UserDefinedDataN ) ; (o) 429
} 430
```

The `DofStorage` child is **optional** (o). When absent, the DOF storage layout is unspecified (`DofStorageNull`), preserving full backward compatibility. 431 432

The same optional child node is added to `DiscreteData_t` (SIDS Section 12.4), using identical structure and semantics (Example 7.8): 433 434

DiscreteData_t (amended, SIDS Section 12.4) 435

```
DiscreteData_t< int CellDimension, int IndexDimension, 437
int VertexSize[IndexDimension], 438
int CellSize[IndexDimension] > := 439
{ 440
... (existing children unchanged) ... 441
+ DofStorage_t DofStorage ; (o) 443
} 444
```

The optional child node is also added to `ZoneSubRegion_t` (SIDS Section 7.9). `ZoneSubRegion_t` holds `DataArray_t` nodes for localized solutions, boundary fluxes, or fields on sub-regions where the shared/independent distinction applies equally (Example 7.9): 445 446 447 448

ZoneSubRegion_t (amended, SIDS Section 7.9) 449

```
ZoneSubRegion_t< int RegionCellDimension > := 451
{ 452
... (existing children unchanged) ... 453
+ DofStorage_t DofStorage ; (o) 455
} 456
```

Finally, the child node is added to `UserDefinedData_t` (SIDS Section 12.10). `UserDefinedData_t` is the standard generic extension point and already accepts `GridLocation_t` and `PointSet` children; permitting `DofStorage_t` there keeps the two consistent and costs nothing, because the position-based API of Section 3.4 covers all parents through a single code path: 457 458 459 460 461

UserDefinedData_t (amended, SIDS Section 12.10) 462

463

```

UserDefinedData_t :=
{
... (existing children unchanged) ...

+   DofStorage_t DofStorage ;      (o)
}

```

3.3.1 Node Placement Rationale

The node is placed as a child of `FlowSolution_t` (rather than, for example, `Zone_t`) because:

- The DOF storage layout is a property of the *solution field*, not of the mesh or the zone.
- A single zone may contain multiple `FlowSolution_t` nodes with different layouts—for example, an HDG solver storing both volume (independent) and trace (shared) solutions, or a multi-physics code storing a shared-DOF displacement field alongside an independent-DOF stress field.
- This parallels the existing `GridLocation_t` child of `FlowSolution_t`, which is also a per-solution-node property.
- Placing it within CPEX 0045’s `SolutionInterpolation_t` would be doubly wrong: that node is a child of `Family_t`, so a per-family setting could not distinguish two solutions on the same family, and it would incorrectly limit the scope to higher-order solutions when the distinction applies at any order.

3.3.2 Why a New Node Rather Than a New `GridLocation_t` Value

The most obvious alternative is to add a `GridLocation_t` value—say `ElementVertex`—meaning “at element-local copies of the vertices.” This proposal rejects that approach:

- **It conflates two orthogonal axes.** `GridLocation_t` answers *where*; DOF storage answers *how many copies*. Each of the seven existing spatial locations can in principle carry either layout, so folding the second axis into the first would require roughly doubling the enumeration (`ElementVertex`, `ElementEdgeCenter`, `ElementFaceCenter`, ...) rather than adding one value.
- **It would not be backward compatible in practice.** `GridLocation_t` is consumed by essentially every CGNS reader, and each has a switch over its values. A new value causes existing readers to fail or fall through to a default, whereas an unknown *child node* is silently ignored by both the MLL and `cgnscheck` (Section 6).

- **The distinction is not always locational.** For `CellCenter` data the layout is independent with no corresponding “element-local” location to name; the flag still carries useful information for coupling frameworks.
- **Precedent.** CGNS already separates orthogonal per-solution properties into sibling child nodes (`GridLocation`, `Rind`, `DataClass`, `DimensionalUnits`) rather than encoding them in a single enumeration.

Note that CPEX 0045 and CPEX 0047 do each add a `GridLocation_t` value (`InterpolationPoints` and `IntegrationPoint`); those additions describe genuinely new *locations*, which is the correct use of that enumeration.

3.4 New MLL Functions

3.4.1 Design: Position-Based API

The API follows the pattern CGNS uses for every other optional scalar metadata child—`GridLocation`, `DataClass`, `Rind`, `DimensionalUnits`, `Descriptor`: a single function pair that operates on the node at the current position, set by `cg_goto`, backed by an internal `cgi_x_address()` helper that resolves the current position to the relevant struct field and enumerates the legal parent labels.

This is preferred over a family of explicit-index functions — one per parent node type, in the style of

```
cg_sol_dof_storage_write(fn,B,Z,S,type)
cg_discrete_dof_storage_write(fn,B,Z,D,type)
cg_subreg_dof_storage_write(fn,B,Z,SR,type)
```

plus matching readers and Fortran bindings — because:

- Two C functions and two Fortran functions cover all four parent node types, instead of six and six.
- Adding a fifth permitted parent later requires one line in the address helper, not two new public functions plus Fortran bindings.
- The address helper locates an existing child *by label* (Section 3.7.1), so no node identifier needs to be cached in the internal structures. This removes a field from three structs and eliminates a class of overwrite bug.
- It matches what users already do for the sibling property: `cg_goto(...); cg_gridlocation_write(CellCenter);`

3.4.2 C API


```

1  /*
2  * Write the DOF storage layout of the node at the current position,
3  * which must have been set by cg_goto()/cg_gopath().
4  *
5  * Valid parent labels: FlowSolution_t, DiscreteData_t,
6  *                      ZoneSubRegion_t, UserDefinedData_t.
7  *
8  * DofStorage - DofStorage_t value. Writing DofStorageNull removes
9  * an existing DofStorage child, restoring the
10 * "unspecified" state.
11 *
12 * Returns CG_OK, or CG_ERROR / CG_INCORRECT_PATH on failure.
13 */
14 CGNSDLL int cg_dof_storage_write(
15     CGNS_ENUMT(DofStorage_t) DofStorage);
16
17 /*
18 * Read the DOF storage layout of the node at the current position.
19 *
20 * DofStorage - [out] DofStorage_t value. Set to DofStorageNull,
21 * with a CG_OK return, when the node has no
22 * DofStorage child (legacy file or unset).
23 *
24 * Returns CG_OK, or CG_ERROR / CG_INCORRECT_PATH on failure.
25 */
26 CGNSDLL int cg_dof_storage_read(
27     CGNS_ENUMT(DofStorage_t) *DofStorage);
28
29 /*
30 * Return the string name for a DofStorage_t value.
31 * Matches the convention of cg_GridLocationName, cg_DataTypeName.
32 * Returns NULL for out-of-range values.
33 */
34 CGNSDLL const char *cg_DofStorageName(
35     CGNS_ENUMT(DofStorage_t) type);

```

Behavior:

531

- `cg_dof_storage_write` creates the `DofStorage` child node under the node at the current position. In `CG_MODE_MODIFY`, an existing `DofStorage` child is deleted first. Writing `DofStorageNull` deletes an existing child without creating a replacement. 532 533 534 535
- `cg_dof_storage_read` returns the value cached in the internal structure by the tree parser. If the node does not exist (legacy file or unset), it returns `CG_OK` with `DofStorageNull`, never an error. 536 537 538
- Both functions return `CG_INCORRECT_PATH` if the current position is not one of the four permitted parent labels, matching `cg_gridlocation_read/_write` behavior. 539 540 541

3.4.3 Fortran API

542

```

1 subroutine cg_dof_storage_write_f(dofStorage, ier)
2   integer, intent(in) :: dofStorage
3   integer, intent(out) :: ier
4
5 subroutine cg_dof_storage_read_f(dofStorage, ier)
6   integer, intent(out) :: dofStorage
7   integer, intent(out) :: ier

```

3.5 Internal Data Structure Changes

543

Four structures in `cgns_header.h` each gain one field: `cgns_sol` (for `FlowSolution_t`), `cgns_discrete` (for `DiscreteData_t`), `cgns_subreg` (for `ZoneSubRegion_t`), and `cgns_user_data` (for `UserDefinedData_t`). 544 545 546

```

1 typedef struct {           /* FlowSolution_t node */
2   /* ... existing fields ... */
3
4   /* CPEX 0050 */
5   CGNS_ENUMT(DofStorage_t) dof_storage;
6 } cgns_sol;
7
8 /* cgns_discrete, cgns_subreg, and cgns_user_data gain the
9  * identically named and typed field. */

```

The `dof_storage` field defaults to `DofStorageNull` (value 0), which is both the zero-initialization value and the “not specified” state. 547 548

No node-identifier field is required. The write path locates an existing child by label at write time (Section 3.7.1), exactly as `cgi_location_address` does for `GridLocation_t`. This mirrors the existing structures, which carry location but no `location_id`. 549 550 551 552

3.6 Internal Lookup Table and String Mapping

553

The string table must be defined in `cgnslib.c` alongside `GridLocationName` and its siblings: 554 555

```

1 const char *DofStorageName[NofValidDofStorage] = {
2   "Null",
3   "UserDefined",
4   "Shared",
5   "Independent"
6 };

```

A string-to-enum helper is added to `cgns_internals.c`, following `cgi_GridLocation` exactly. That includes its forward-compatibility behavior: an unrecognized string in a file written by a *newer* library version degrades to `UserDefined` with a warning rather than failing. 556 557 558 559

```

1 int cgi_DofStorage(char *Name, CGNS_ENUMT(DofStorage_t) *type)
2 {
3     int i;
4     for (i = 0; i < NofValidDofStorage; i++) {
5         if (strcmp(Name, DofStorageName[i]) == 0) {
6             (*type) = (CGNS_ENUMV(DofStorage_t))i;
7             return CG_OK;
8         }
9     }
10    if (cg->version > CGNSLibVersion) {
11        (*type) = CGNS_ENUMV(DofStorageUserDefined);
12        cgi_warning("Unrecognized DofStorage Type '%s' "
13                  "replaced with 'UserDefined'", Name);
14        return CG_OK;
15    }
16    cgi_error("Unrecognized DofStorage: %s", Name);
17    return CG_ERROR;
18 }

```

3.7 MLL Implementation Requirements

560

Throughout this section, `cg` is the file-scope current-file pointer used internally by `cgnslib.c`, and `posit` is the file-scope current-position record maintained by `cg_goto`. Neither is declared in the listings, matching the surrounding library code.

561

562

563

3.7.1 Address Resolution Function (`cgns_internals.c`)

564

```

1 CGNS_ENUMT(DofStorage_t) *cgi_dof_storage_address(
2     int local_mode, int *ier)
3 {
4     double *id, parent_id;
5     CGNS_ENUMT(DofStorage_t) *dof_storage = 0;
6     int nnod;
7
8     if (posit == 0) {
9         cgi_error("No current position set by cg_goto\n");
10        (*ier) = CG_ERROR;
11        return CG_OK;
12    }
13
14    /* Possible parents for DofStorage_t: FlowSolution_t,
15     * DiscreteData_t, ZoneSubRegion_t, UserDefinedData_t */
16    if (strcmp(posit->label, "FlowSolution_t") == 0)
17        ADDRESS4SINGLE_ALLOC(cgns_sol, dof_storage)
18    else if (strcmp(posit->label, "DiscreteData_t") == 0)
19        ADDRESS4SINGLE_ALLOC(cgns_discrete, dof_storage)
20    else if (strcmp(posit->label, "ZoneSubRegion_t") == 0)
21        ADDRESS4SINGLE_ALLOC(cgns_subreg, dof_storage)
22    else if (strcmp(posit->label, "UserDefinedData_t") == 0)
23        ADDRESS4SINGLE_ALLOC(cgns_user_data, dof_storage)
24    else {
25        cgi_error("DofStorage_t node not supported under "
26                  "'%s' type node", posit->label);
27        (*ier) = CG_INCORRECT_PATH;
28        return CG_OK;
29    }
30 }

```

```

29     }
30
31     /* On modify, delete any existing child, located by label */
32     if (cg->mode == CG_MODE_MODIFY &&
33         local_mode == CG_MODE_WRITE) {
34         if (cgi_get_nodes(parent_id, "DofStorage_t", &nnod, &id))
35             return CG_OK;
36         if (nnod > 0) {
37             if (cgi_delete_node(parent_id, id[0])) {
38                 (*ier) = CG_ERROR;
39                 return CG_OK;
40             }
41             CGNS_FREE(id);
42         }
43     }
44     return dof_storage;
45 }

```

Locating the existing child by label is what makes a cached node identifier unnecessary, 565
and is why the overwrite path cannot create a duplicate child. 566

3.7.2 Internal Tree Parser Update (cgns_internals.c) 567

When a CGNS file is opened, the MLL tree parser crawls the HDF5/CGIO tree and 568
populates internal C structures. The parser must be updated to recognize child nodes 569
with label `DofStorage_t` during the traversal of `FlowSolution_t` (`cgi_read_sol`), 570
`DiscreteData_t` (`cgi_read_discrete`), `ZoneSubRegion_t` (`cgi_read_subregion`), 571
and `UserDefinedData_t` (`cgi_read_user_data`). 572

The required behavior is: 573

1. During memory allocation of the parent structure, the `dof_storage` field is 574
zero-initialized, which corresponds to `DofStorageNull`. 575
2. During the child-node traversal, if a child with label `"DofStorage_t"` 576
is found, read its C1 string data and map it to the enum value via 577
`cgi_DofStorage(string_data, &dof_storage)`. 578
3. If no such child node exists, the field retains its default `DofStorageNull` value. 579

This follows the existing pattern used for `GridLocation_t` parsing in `cgi_read_sol`. 580
No node identifier is stored; see Section 3.7.1. 581

3.7.3 Read Function Implementation 582

```

1 int cg_dof_storage_read(CGNS_ENUMT(DofStorage_t) *DofStorage)
2 {
3     CGNS_ENUMT(DofStorage_t) *dof_storage;
4     int ier = 0;
5
6     CHECK_FILE_OPEN
7
8     if (cgi_check_mode(cg->filename, cg->mode, CG_MODE_READ))
9         return CG_ERROR;
10
11     dof_storage = cgi_dof_storage_address(CG_MODE_READ, &ier);
12     if (dof_storage == 0) return ier;
13
14     (*DofStorage) = (*dof_storage);
15     return CG_OK;
16 }

```

Because the tree parser already handles the absent-node case by leaving the field at `DofStorageNull`, the read function simply returns the cached value and `CG_OK`. This avoids any need to intercept `CG_NODE_NOT_FOUND` at read time: the absent-node case is handled once during file open, and all subsequent reads are served from the in-memory cache. Legacy code that iterates over solutions and queries the DOF storage layout will always receive `CG_OK` with `DofStorageNull` for files that predate this CPEX.

3.7.4 Write Function Implementation

```

1 int cg_dof_storage_write(CGNS_ENUMT(DofStorage_t) DofStorage)
2 {
3     CGNS_ENUMT(DofStorage_t) *dof_storage;
4     int ier = 0;
5     cgsizet dim_vals;
6     double posit_id, dummy_id;
7
8     CHECK_FILE_OPEN
9
10    if (cgi_check_mode(cg->filename, cg->mode, CG_MODE_WRITE))
11        return CG_ERROR;
12
13    if (INVALID_ENUM(DofStorage, NofValidDofStorage)) {
14        cgi_error("Invalid DofStorage value: %d", (int)DofStorage);
15        return CG_ERROR;
16    }
17
18    /* Resolves the parent, and on modify deletes any
19     * existing DofStorage child (located by label). */
20    dof_storage = cgi_dof_storage_address(CG_MODE_WRITE, &ier);
21    if (dof_storage == 0) return ier;
22
23    (*dof_storage) = DofStorage;
24
25    /* Null means "unspecified": record nothing on disk. Any
26     * pre-existing child was removed by the address helper. */
27    if (DofStorage == CGNS_ENUMV(DofStorageNull)) return CG_OK;

```

```

28
29     if (cgi_posit_id(&posit_id)) return CG_ERROR;
30     dim_vals = (cgsz_t)strlen(DofStorageName[DofStorage]);
31     if (cgi_new_node(posit_id, "DofStorage", "DofStorage_t",
32                     &dummy_id, "C1", 1, &dim_vals,
33                     (void *)DofStorageName[DofStorage]))
34         return CG_ERROR;
35
36     return CG_OK;
37 }

```

Two details are worth noting. First, validation uses the library’s own `INVALID_ENUM(E,EMAX)` macro from `cgns_header.h`, as `cg_gridlocation_write` does, rather than an open-coded `switch`. Second, no node identifier is stored after `cgi_new_node` returns, because the address helper will find the node by label on any subsequent write.

3.8 Data Layout for Independent DOFs at Shared Mesh Vertices

A common and important case arises when a DG solver uses a mesh whose vertices are shared in the standard CGNS sense—enabling mesh walking, stream tracing, and connectivity queries via the standard `Elements_t` tables—but stores element-local, independent solution values at each element’s copy of those vertices. At a shared vertex adjacent to k elements there are k distinct solution values, one per element; readers must not average or conflate them.

The correct CGNS encoding for this pattern is `GridLocation_t = Vertex` combined with `DofStorage_t = Independent` and a `PointList` that lists every element-vertex pair explicitly (repeated vertex indices are expected and meaningful). The `DofStorage_t` node is the machine-readable flag that tells readers these repeated entries represent independent DOFs, not duplicate data.

Without the `DofStorage_t` node, a reader cannot distinguish this layout from a malformed shared-DOF solution with duplicate entries, which is why the node is necessary. With it, a post-processor or coupling framework can correctly:

- walk the mesh using the shared-vertex connectivity for geometry operations (stream tracing, surface intersection, etc.);
- read solution values element-locally, preserving the jumps at shared interfaces.

3.8.1 Applicability and Restrictions

This encoding is subject to the following restrictions, which the SIDS amendment in Section 8 makes normative.

Unstructured zones only. The DOF ordering is defined by traversal of the zone’s `Elements_t` sections, which structured zones do not have. The encoding is

therefore valid only for `ZoneType_t = Unstructured`. Structured zones have no way to express independent DOFs at shared vertices, and this CPEX does not add one. A structured solver with element-local DOFs should either store cell-centered data (`GridLocation_t = CellCenter`, which is unambiguously independent) or convert the affected region to an unstructured zone. Note that CPEX 0045’s `InterpolationOrders` child is likewise “recognized only for `Unstructured` zones,” so the high-order route is not a structured fallback either.

The PointList must cover every element of the zone. The ordering rule below is a traversal of the zone’s elements; a `PointList` covering only some elements has no defined anchor into that traversal. A partial-coverage independent solution should use `GridLocation_t = InterpolationPoints` (CPEX 0045) or `IntegrationPoint` (CPEX 0047), both of which carry explicit per-element element-index lists.

Element type support. Fixed-size element types and `MIXED` and `NGON_n` sections are supported. `NFACE_n` is *not*: a polyhedral element’s vertices are defined only indirectly, through its constituent faces, and `CGNS` defines no canonical vertex ordering for such an element, so “the element’s vertices in order” is not well defined. Zones containing `NFACE_n` sections must use `GridLocation_t = InterpolationPoints` or `IntegrationPoint` for independent DOFs.

Rind_t must not be combined with this encoding. The `PointList` already enumerates every DOF explicitly, so there is no separate `rind` extent to declare, and the `SIDS` defines no ordering for `rind` entries within a point set. `Rind_t` remains fully usable with `DofStorage_t = Independent` when no point set is present; see Section 3.9.

Storage overhead. Because the `PointList` is, by construction, the concatenation of the zone’s element connectivity, it duplicates data already present in the `Elements_t` sections at a cost of one `cgsize_t` per DOF. This is exactly the conflation of `dof`-relation and `topo`-relation that Miller [8] describes. The overhead should be weighed honestly against the two alternatives.

Writing the solution at `InterpolationPoints` instead does *not* destroy the shared-vertex connectivity: `GridCoordinates_t` and `Elements_t` are untouched, and mesh walking continues to work. On capability alone, therefore, the 0045 route also satisfies the requirement of Section 2.2, and this proposal should not claim otherwise. What it offers over that route is a far smaller implementation burden for readers—no `Family_t`-level `SolutionInterpolation_t` lookup, no basis evaluation, no control-point machinery—for the common case in which the DOF positions simply *are* the mesh vertices. For first-order DG that distinction is the difference between reading one extra scalar node and implementing an interpolation framework.

The alternative actually observed in production, by contrast, is strictly worse than either: duplicating the grid coordinates (Section 2.2) inflates `GridCoordinates_t`

by the average vertex valence—far more than one `cgsizes_t` per DOF—and destroys the connectivity outright. Measured against that, the `PointList` redundancy is a substantial improvement.

Where CPEX 0045 or CPEX 0047 has been adopted and the DOF positions are not the mesh vertices, `InterpolationPoints` or `IntegrationPoint` remains the preferred encoding: both size the array from per-element DOF counts with no redundant index list. It should also be acknowledged that community feedback has objected to *existing* cases where an identity point set must be written purely as indirection; the redundancy here is a real cost to weigh, not a formality.

3.8.2 Other Grid Locations

For element-local DOFs at non-vertex positions (interior quadrature points, face quadrature points, etc.) under `FlowSolution_t`, use `GridLocation_t = InterpolationPoints` with CPEX 0045 metadata, or `GridLocation_t = IntegrationPoint` with CPEX 0047 metadata. Both are restricted by their respective proposals: CPEX 0045 states that `InterpolationPoints` “is valid only on `FlowSolution_t` nodes,” and CPEX 0047 permits `IntegrationPoint` under `FlowSolution_t`, `ZoneSubRegion_t`, `BC_t`, and `BCDataSet_t`. Consequently `DiscreteData_t` has no standard encoding for element-local DOFs at non-vertex positions; `DofStorage_t = Independent` on a `DiscreteData_t` node is meaningful for `CellCenter` data, and for vertex data under the `PointList` encoding above, but not otherwise. Closing that gap is out of scope here.

`GridLocation_t = Vertex` with a `PointList` is specifically the correct choice when the DOF positions coincide with mesh vertices and the shared-vertex mesh connectivity is to be preserved.

`GridLocation_t = Vertex` with `Independent` and *no* `PointList` is not valid: the SIDS `DataSize` formula for `Vertex` yields `DataSize = VertexSize` (the shared-vertex count), which cannot represent the larger element-local DOF count. Similarly, `PointRange` is not suitable for this case because a contiguous range of vertex indices cannot express the element-local repetition of shared vertices; only `PointList` provides the necessary per-element DOF ordering. `GridLocation_t = CellCenter` with `Independent` remains valid since cell-centered data is inherently one value per element with no shared-entity ambiguity.

3.8.3 DOF Ordering Rule

SIDS Amendment — `PointList` semantics for independent DOFs. When `DofStorage_t` is `Independent` and a `PointList` is present, with `GridLocation_t = Vertex`, under any of `FlowSolution_t`, `DiscreteData_t`, `ZoneSubRegion_t`, or `UserDefinedData_t`, repeated vertex indices are permitted and represent element-

local DOFs. Solution array position i corresponds to the i -th entry of the `PointList`, and the `PointList` shall be constructed as follows:

1. Traverse the zone's elements in ascending **element index** order. Element indices are the global, zone-wide indices defined by each `Elements_t` section's `ElementRange`; they are *not* the section storage order, and sections are not required to be stored in ascending `ElementRange` order.
2. Within each element, list the element's vertex indices in the canonical vertex order defined for its element type in SIDS §7.3.
3. For `MIXED` sections, the element-type code that precedes each element's vertex list in the connectivity array is not a DOF and is not listed.
4. For `NGON_n` sections, list the face's vertex indices in connectivity order; the per-element vertex count varies and the `PointList` length is the sum over all elements.

Repeated indices under `DofStorage_t = Shared`, `DofStorageUserDefined`, or `DofStorageNull` remain undefined and invalid. This amendment does not affect `PointList` semantics under any other condition.

Because the `PointList` must equal the traversal above, a validator can check it exactly against the `Elements_t` connectivity; see Section 11.

See Example 7.7 for the complete C encoding of this pattern.

3.9 Interaction with `Rind_t` (Ghost Layers)

When no point set is present, `DofStorage_t` applies uniformly to the entire solution field, including any rind (ghost) entries declared by `Rind_t`. An `Independent` label means that all data entries—interior and rind alike—are element-local: each rind entry belongs to one element and must not be interpreted as a shared vertex value. DG solvers that use ghost layers for MPI halo exchange may therefore store element-local rind DOFs in the standard way; the `Independent` label covers them. Rind entries of a `Shared` solution retain their usual shared-DOF semantics. No special treatment of the rind is required beyond what `Rind_t` already specifies; the DOF storage label is consistent across the full data array.

As stated in Section 3.8.1, `Rind_t` must not be combined with the `Vertex + Independent + PointList` encoding, because the SIDS defines no position for rind entries within a point set.

4 Interaction with CPEX 0045 (Higher-Order Elements)

CPEX 0045 [4] introduced `GridLocation_t = InterpolationPoints`, the `InterpolationOrders` and `CharacteristicLength` children of `FlowSolution_t`, and the `SolutionInterpolation_t` and `ElementInterpolation_t` children of `Family_t`. The present CPEX complements rather than modifies that infrastructure:

- CPEX 0045 answers: “What polynomial basis and order describes the solution within each element?”
- This CPEX answers: “Are the resulting DOFs shared across element interfaces or element-local?”

Both pieces of metadata are needed for a complete description of a higher-order solution field. They are orthogonal:

	Shared	Independent
Lagrange $p = 1$	CG FEM [‡]	DG $p = 1$
Lagrange $p = 3$	CG spectral [‡]	DG spectral
Hierarchical $p = 2$	n/a [§]	DG (modal)
$p = 0$ (constant)	n/a [†]	FV / DG $p = 0$

[†] A $p = 0$ basis has no interface DOFs to share: its single per-element DOF has no geometric support on the element boundary. **Shared** is therefore not merely unusual at $p = 0$ but meaningless, and `cgnscheck` should warn on it (Section 11).

[‡] Realizable only at `GridLocation_t = Vertex` on the predefined element types, where the `Elements_t` connectivity supplies the DOF identification. It is *not* realizable under `InterpolationPoints`, where **Shared** is invalid on a volume block (Section 4.2). Since predefined element types reach only fourth order, shared storage is expressible only for $p \leq 4$.

[§] Hierarchical bases admitting a vertex/edge/face/interior mode decomposition support continuous assembly mathematically, but CPEX 0045’s monomial encodings (`ParametricMonomialsPascal`, `CartesianMonomialsPascal`) store modal coefficients that have no geometric support on any sub-entity, and 0045 fixes no sub-entity ordering for them. There is consequently no entity to share and no DOF correspondence to establish, so **Shared** cannot be acted upon. The same holds for `ParametricLagrange` with user-supplied control points: 0045 guarantees a principal-vertices-first ordering only for `ElementInterpolation_t`, not for `SolutionInterpolation_t`, and defines no orientation-aware correspondence between the points of two adjacent elements.

4.1 Recommended Practice

For higher-order solutions using `GridLocation_t = InterpolationPoints`:

1. Use CPEX 0045 to define the element type, interpolation basis, and polynomial order via `SolutionInterpolation_t` on the `Family_t` and

`InterpolationOrders` on the `FlowSolution_t`. 759

2. Use this CPEX to set `DofStorage_t` on the `FlowSolution_t` node. 760

Examples 7.2 and 7.4 show both steps. 761

For lower-order solutions using `GridLocation_t = Vertex` or `CellCenter`, this 762
CPEX can be used independently of CPEX 0045; see Examples 7.1 and 7.6. See 763
Section 3.8 for the prescribed data layout when independent DOFs coincide with 764
shared mesh vertices. 765

4.2 `DofStorage_t` Under `InterpolationPoints` 766

Under `GridLocation_t = InterpolationPoints`, CPEX 0045 defines the field- 767
array length as 768

$$L = \sum_{e \in \mathcal{E}} N_{\text{DOFs}}(e),$$

where $\mathcal{E}$ is the set of elements covered by the block—all zone elements when no 769
`PointRange/PointList` is present, or the listed elements otherwise—with each 770
element’s DOFs stored contiguously. Because the array is a concatenation of per- 771
element DOF blocks, the following rules apply: 772

- `DofStorage_t = Independent` is the layout’s natural reading and requires no 773
additional metadata. It is the expected value for volume DG and spectral-element 774
blocks. 775
- `DofStorage_t = Shared` is permitted *only* when the block’s element set con- 776
sists of interface (face or edge) elements, listed explicitly by a `PointList` 777
or `PointRange` over the element indices of the corresponding face or edge 778
`Elements_t` section(s). The shared reading is then: each listed interface ele- 779
ment’s DOF block is shared between the volume elements adjacent to it. This 780
is the HDG trace case (Example 7.4). 781
- `DofStorage_t = Shared` on a block whose element set is the zone’s volume 782
elements—in particular, any `InterpolationPoints` block with *no* point set—is 783
invalid. Under that sizing each interface DOF would appear once per adjacent 784
volume element, which contradicts the definition of `Shared`. `cgnscheck` should 785
report this as an error (Section 11). 786

5 Interaction with CPEX 0047 (Integration Points) 787

CPEX 0047 [5] introduces `GridLocation_t = IntegrationPoint`, quadrature-rule 788
definitions, and an `ItgPointsStartOffset` array giving the starting position of 789
each element’s integration-point data within the solution array. It permits that grid 790
location under `FlowSolution_t`, `ZoneSubRegion_t`, `BC_t`, and `BCDataSet_t`. 791

The two proposals are complementary and do not conflict:

- **CPEX 0047 supplies an explicit dof-map for one grid location.** `ItsPointsStartOffset` is precisely the per-element offset array whose absence motivates Section 3.8. Where it applies, it is a better mechanism than the redundant `PointList` encoding, and this CPEX recommends it over that encoding.
- **IntegrationPoint data is inherently independent.** Integration points are interior to elements and the offset array assigns a disjoint slice of the array to each element, so `DofStorage_t = Independent` is the only valid value under `GridLocation_t = IntegrationPoint`. Recording it remains useful for uniform reader dispatch, but adds no information a reader could not derive; `Shared` under this location should be reported as an error by `cgnscheck`.
- **CPEX 0047 does not subsume this proposal.** It says nothing about vertex-, edge-, or face-located DOFs, which is the only place a storage discriminator is load-bearing, and it does not cover `DiscreteData_t`.
- **Boundary data.** CPEX 0047 extends `BC_t` and `BCDataSet_t`; this CPEX deliberately does not (Design Decision 1), and its `IntegrationPoint` location already covers element-local boundary data, including the weakly-imposed boundary conditions of continuous methods.

One editorial concern spans the two proposals: CPEX 0045's `InterpolationPoints` and CPEX 0047's `IntegrationPoint` are similar names for different `GridLocation_t` values. That is a matter for the editors of those proposals, but readers of this one should note the distinction.

6 Backward Compatibility

- **Existing files:** Fully compatible. The `DofStorage` node is optional; its absence is equivalent to `DofStorageNull`. No existing file is invalidated.
- **Existing readers:** Unaffected. Per the CGNS convention, readers silently ignore unknown child nodes. An older library reading a file with `DofStorage` nodes will simply skip them. `cgnscheck` validates only the child labels it knows about and does not enumerate or warn on unrecognized ones, so existing validation runs are unaffected as well.
- **Existing writers:** Unaffected. Codes that do not call the new API functions produce files without `DofStorage` nodes, identical to the current behavior.
- **Newer-to-older direction:** A file written by a future library with an additional `DofStorage_t` value will be read by a CPEX-0050-era

library as `DofStorageUserDefined` with a warning, not an error, because `cgi_DofStorage` follows `cgi_GridLocation`'s forward-compatibility rule.

- **ABI:** The `cgns_sol`, `cgns_discrete`, `cgns_subreg`, and `cgns_user_data` structures each grow by one `int`-sized enum field. This is an ABI change requiring recompilation of code that directly accesses the internal structures. The public API is purely additive (new functions, new enum), so no existing function signatures change.

6.1 Parallel I/O

No new `cgp_*` functions are required, but the constraint under which the serial functions may be used in a parallel program must be stated explicitly.

The parallel MLL (`pcgnslib.c`) contains no handling of `GridLocation`, `DataClass`, or any other scalar metadata child; such nodes are written by the serial MLL even in parallel programs. `DofStorage` follows suit. Because writing it creates—and, on modify, deletes—an HDF5 group, and HDF5 structural modification under MPI-IO is a collective operation:

- `cg_dof_storage_write` **shall** be called by all ranks in the communicator, with identical arguments and from an identical current position.
- The same applies to the `cg_goto` call that establishes the position.
- `cg_dof_storage_read` may be called independently once the file is open, since it is served entirely from the in-memory structure populated at open time.

This is the same rule that already governs `cg_sol_write`, `cg_gridlocation_write`, and every other metadata-creating serial call used alongside `cgp_*` data transfers; it is stated here because the proposal adds a new call subject to it.

6.2 Interpreting Legacy Data (`DofStorageNull`)

`DofStorageNull` means *unspecified*, not “shared.” A reader must not silently substitute a default. The following inference rules are recommended—non-normatively—for readers that must nevertheless act on files predating this CPEX:

CellCenter, FaceCenter, EdgeCenter, [IJK]FaceCenter Treat as Independent.
There is exactly one value per entity and no shared-entity ambiguity.

Vertex with DataSize = VertexSize Treat as Shared. Before this CPEX, the SIDS `DataSize` formula admitted no other layout at this size.

Vertex with a PointList Unspecified. A legacy file gives no way to tell an independent-DOF layout from a genuine vertex subset. Readers should surface the ambiguity rather than guess.

InterpolationPoints (CPEX 0045) Treat as Independent, per Section 4.2. 861

IntegrationPoint (CPEX 0047) Treat as Independent. 862

Note that these rules make no “assume shared for backward compatibility” blanket 863
 assumption: since most CFD files are cell-centered finite volume, such an assumption 864
 would be wrong for the majority of existing data. Example 7.10 shows a restart 865
 reader applying this dispatch. 866

7 Examples 867

All examples use the position-based API: `cg_goto` to the target node, then 868
`cg_dof_storage_write`. Error checking is omitted for brevity; production code 869
 must check every return value. 870

The examples serve two different purposes, and it is worth separating them. Exam- 871
 ple 7.7 is the motivating case of Section 2.2: the encoding it shows has no alternative 872
 today, and it is the reason for this proposal. Example 7.1 is its counterpart, and 873
 Example 7.3 shows the reader-side dispatch that makes the pair useful. The remain- 874
 der are *illustrative*: each shows how to label a layout that is already determined by 875
 the grid location and the sizing rules, so a reader ignoring the node would reach the 876
 same conclusion. They are retained to document correct usage across the permitted 877
 parent nodes, not as evidence that the node is required in those situations—per 878
 Section 2.4, it is not. 879

7.1 Example 1: Writing a Shared-DOF (CG) Solution 880

```

1  /* Create a vertex-located solution node */
2  int S, F;
3  cg_sol_write(fn, B, Z, "FlowSolution", CGNS_ENUMV(Vertex), &S);
4
5  /* Mark DOFs as shared (CG FEM) */
6  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S, "end");
7  cg_dof_storage_write(CGNS_ENUMV(Shared));
8
9  /* Write solution data (one value per mesh vertex, shared) */
10 cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble),
11               "Pressure", pressure, &F);

```

7.2 Example 2: Writing an Independent-DOF (DG) Solution 881

```

1  /* Create an interpolation-point-located solution node.
2  * Per CPEX 0045, a block covering the whole zone at uniform
3  * order needs no point set; DataSize is the sum of the
4  * per-element DOF counts over all zone elements. */
5  int S, F;
6  cg_sol_write(fn, B, Z, "FlowSolution_DG",
7             CGNS_ENUMV(InterpolationPoints), &S);
8
9  /* Set polynomial order (CPEX 0045):
10 * spatialOrder = 3, temporalOrder = 0 */
11 cg_sol_interpolation_order_write(fn, B, Z, S, 3, 0);
12
13 /* Mark DOFs as element-local */
14 cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S, "end");
15 cg_dof_storage_write(CGNS_ENUMV(Independent));
16
17 /* Write element-local DOFs */
18 cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble),
19             "Pressure", pressure_dg, &F);

```

7.3 Example 3: Reading and Dispatching on DOF Storage Layout

882

```

1  CGNS_ENUMT(DofStorage_t) layout;
2
3  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S, "end");
4  cg_dof_storage_read(&layout);
5
6  switch (layout) {
7  case CGNS_ENUMV(Shared):
8      /* Shared DOFs: interpolate using global mesh connectivity */
9      visualize_shared(fn, B, Z, S);
10     break;
11
12  case CGNS_ENUMV(Independent):
13      /* Element-local DOFs: render per-element, preserve jumps */
14      visualize_independent(fn, B, Z, S);
15     break;
16
17  case CGNS_ENUMV(DofStorageUserDefined): {
18      /* Writer must have left a DofStorageDescription
19       * descriptor identifying the convention. */
20      char dname[CGIO_MAX_NAME_LENGTH+1];
21      char *text = NULL;
22      int nd, n;
23      cg_ndescriptors(&nd);
24      for (n = 1; n <= nd; n++) {
25          cg_descriptor_read(n, dname, &text);
26          if (text == NULL) continue;
27          if (strcmp(dname, "DofStorageDescription") == 0)
28              visualize_user_defined(fn, B, Z, S, text);
29          cg_free(text);
30          text = NULL;
31      }
32     break;
33 }
34
35 default:

```

```

36  /* DofStorageNull: legacy file. Apply the inference
37  * rules of Section 6.2 -- do not assume Shared. */
38  visualize_legacy(fn, B, Z, S);
39  break;
40  }

```

7.4 Example 4: HDG Solver with Both DOF Storage Types

883

An HDG solver stores element-local volume DOFs and shared trace DOFs on the element skeleton. The trace field is defined on *face* elements, so the file must contain a face `Elements_t` section and the trace solution must reference it through a point set. Without that point set the array would be sized over the zone's volume elements, which is invalid (Section 4.2).

884

885

886

887

888

```

1  /* ---- Volume DOFs (independent) ---- */
2  int S_vol, F;
3  cg_sol_write(fn, B, Z, "VolumeSolution",
4  CGNS_ENUMV(InterpolationPoints), &S_vol);
5  cg_sol_interpolation_order_write(fn, B, Z, S_vol, 3, 0);
6  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S_vol, "end");
7  cg_dof_storage_write(CGNS_ENUMV(Independent));
8
9  /* ---- Face elements defining the skeleton ----
10  * Faces occupy element indices firstFace..lastFace. */
11  int Sec;
12  cg_section_write(fn, B, Z, "Faces", CGNS_ENUMV(TRI_3),
13  firstFace, lastFace, 0, face_conn, &Sec);
14
15  cgsize_t faceRange[2];
16  faceRange[0] = firstFace;
17  faceRange[1] = lastFace;
18
19  /* ---- Case A: p=0 traces, one DOF per face ----
20  * FaceCenter in a 3-D zone has no SIDS DataSize formula, so a
21  * point set over face element indices is REQUIRED; cg_sol_write
22  * alone would leave the field size undefined. */
23  int S_trace;
24  cg_sol_ptset_write(fn, B, Z, "TraceSolution",
25  CGNS_ENUMV(FaceCenter), CGNS_ENUMV(PointRange),
26  2, faceRange, &S_trace);
27  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S_trace, "end");
28  cg_dof_storage_write(CGNS_ENUMV(Shared));
29  cg_field_write(fn, B, Z, S_trace, CGNS_ENUMV(RealDouble),
30  "TracePressure", trace_p, &F);
31
32  /* ---- Case B: higher-order traces ----
33  * InterpolationPoints with a point set listing the FACE element
34  * indices. Each listed face contributes N_DOFs(face) values, and
35  * each face's block is shared by its two adjacent volume elements,
36  * so Shared is the correct label (Section 4.2). */
37  int S_trace_ho;
38  cg_sol_ptset_write(fn, B, Z, "TraceSolution_H0",
39  CGNS_ENUMV(InterpolationPoints), CGNS_ENUMV(PointRange),
40  2, faceRange, &S_trace_ho);
41  cg_sol_interpolation_order_write(fn, B, Z, S_trace_ho, 2, 0);
42  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S_trace_ho, "end");

```



```

43 cg_dof_storage_write(CGNS_ENUMV(Shared));
44 cg_field_write(fn, B, Z, S_trace_ho, CGNS_ENUMV(RealDouble),
45             "TracePressure", trace_p_ho, &F);

```

7.5 Example 5: Fortran Usage

889

```

1  integer :: S, ier, layout
2
3  ! Write a shared-DOF solution
4  call cg_sol_write_f(fn, B, Z, 'FlowSolution', Vertex, S, ier)
5  call cg_goto_f(fn, B, ier, 'Zone_t', Z, 'FlowSolution_t', S, 'end')
6  call cg_dof_storage_write_f(Shared, ier)
7
8  ! Read the DOF storage layout back
9  call cg_goto_f(fn, B, ier, 'Zone_t', Z, 'FlowSolution_t', S, 'end')
10 call cg_dof_storage_read_f(layout, ier)
11 if (layout == Independent) then
12     ! Element-local interpolation
13 end if

```

7.6 Example 6: Low-Order Finite Volume (DG $p = 0$)

890

```

1  /* Cell-centered FV solution */
2  int S;
3  cg_sol_write(fn, B, Z, "FlowSolution",
4             CGNS_ENUMV(CellCenter), &S);
5
6  /* Explicitly record the independent layout */
7  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S, "end");
8  cg_dof_storage_write(CGNS_ENUMV(Independent));

```

A `CellCenter` solution is already unambiguously independent, so a reader could derive this without the tag. Recording it anyway is recommended but optional: it lets a coupling framework or post-processor dispatch on one uniform query across all solution nodes in a file, instead of maintaining a special case for each grid location. The tag is genuinely load-bearing—rather than merely tidy—in the cases of Examples 7.4, 7.7, 7.11, and 7.12.

7.7 Example 7: Independent DOFs on a Shared-Vertex Mesh

897

This example addresses the case where the mesh uses shared vertices (enabling stream tracing and mesh walking via standard `Elements_t` connectivity), but each element stores its own independent solution values at those vertices. The `DofStorage_t` node is the flag that tells readers the repeated vertex indices in the `PointList` are independent DOFs, not duplicates. The zone must be unstructured, the `PointList` must cover every element, and the traversal order is fixed by Section 3.8.3.

```

1  /*
2  * Given, for a single-section unstructured zone:
3  *   cgsizet_t nVerts;      shared vertices in the zone
4  *   cgsizet_t nElems;      elements in the zone
5  *   int       nVertsPerElem; vertices per element (fixed type)
6  *   cgsizet_t *elem_conn;   connectivity, nElems*nVertsPerElem,
7  *                           1-based, ascending element index
8  *   double    *pressure_dg; nElems*nVertsPerElem values
9  *
10 * Total DOFs = nElems * nVertsPerElem (not nVerts).
11 */
12 cgsizet_t nDofs = nElems * (cgsizet_t)nVertsPerElem;
13 cgsizet_t *point_list = (cgsizet_t *)malloc(nDofs * sizeof(cgsizet_t));
14 if (point_list == NULL) return 1;
15
16 /* The PointList is the connectivity traversed in ascending
17 * element index order, each element's vertices in canonical
18 * order. For this single fixed-size section that is exactly
19 * the connectivity array; a multi-section, MIXED, or NGON_n
20 * zone requires the traversal of Section 3.8.3. */
21 {
22     cgsizet_t i;
23     for (i = 0; i < nDofs; i++)
24         point_list[i] = elem_conn[i];
25 }
26
27 /* Create the solution node with a PointList. Vertex location
28 * preserves shared-mesh connectivity for geometry operations,
29 * while DataSize becomes nDofs rather than nVerts. */
30 int S, F;
31 cg_sol_ptset_write(fn, B, Z, "FlowSolution_DG",
32                  CGNS_ENUMV(Vertex), CGNS_ENUMV(PointList),
33                  nDofs, point_list, &S);
34
35 /* Mark as independent: values at repeated vertex indices are
36 * independent across elements, ordered by ascending element
37 * index then canonical vertex order. */
38 cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S, "end");
39 cg_dof_storage_write(CGNS_ENUMV(Independent));
40
41 /* Write element-local solution data */
42 cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble),
43               "Pressure", pressure_dg, &F);
44
45 free(point_list);

```

A reader distinguishes this from a shared-DOF solution as follows:

904

```

1  CGNS_ENUMT(DofStorage_t) layout;
2
3  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S, "end");
4  cg_dof_storage_read(&layout);
5
6  if (layout == CGNS_ENUMV(Independent)) {
7      /* PointList entries are element-local DOF indices.
8       * Values at the same vertex index from different elements
9       * are independent -- do not average. */
10 }

```

7.8 Example 8: DiscreteData_t with an Independent Error Indicator 905

This example shows a DG error indicator stored in a DiscreteData_t node. The 906
usage is identical to FlowSolution_t: only the cg_goto path differs. 907

```

1  /* Create a cell-centered discrete data node for error indicators */
2  int D;
3  cg_discrete_write(fn, B, Z, "ErrorIndicator", &D);
4
5  cg_goto(fn, B, "Zone_t", Z, "DiscreteData_t", D, "end");
6  cg_gridlocation_write(CGNS_ENUMV(CellCenter));
7
8  /* Mark as independent (element-local error estimates) */
9  cg_dof_storage_write(CGNS_ENUMV(Independent));
10
11 /* Write per-element error data */
12 cg_array_write("ElementError", CGNS_ENUMV(RealDouble),
13               1, &nElems, error_data);

```

Note that a single cg_goto serves all three calls: neither cg_gridlocation_write, 908
cg_dof_storage_write, nor cg_array_write moves the current position. 909

7.9 Example 9: ZoneSubRegion_t with Boundary Flux Data 910

This example stores independent boundary flux data on a ZoneSubRegion_t that 911
references a boundary condition region. 912

```

1  /* The sub-region dimension is one less than the zone's cell
2   * dimension, so query the base rather than hard-coding it. */
3  char basename[CGIO_MAX_NAME_LENGTH+1];
4  int cellDim, physDim;
5  cg_base_read(fn, B, basename, &cellDim, &physDim);
6
7  /* Create a sub-region referencing a BC */
8  int SR;
9  cg_subreg_bcname_write(fn, B, Z, "WallFluxes",
10                        cellDim - 1, "Wall_BC", &SR);
11
12 /* Mark as independent (each face owns its own flux values) */
13 cg_goto(fn, B, "Zone_t", Z, "ZoneSubRegion_t", SR, "end");
14 cg_dof_storage_write(CGNS_ENUMV(Independent));
15
16 /* Write flux data on the sub-region */
17 cg_array_write("HeatFlux", CGNS_ENUMV(RealDouble),
18               1, &nBCFaces, heat_flux);

```

7.10 Example 10: Solver Restart with DOF-Storage-Aware Read 913

A solver reading a restart file must reconstruct the solution state correctly based on 914
the DOF storage layout. 915

```

1  /* Read solution metadata */
2  char sol_name[CGIO_MAX_NAME_LENGTH+1];
3  CGNS_ENUMT(GridLocation_t) loc;
4  cg_sol_info(fn, B, Z, S, sol_name, &loc);
5
6  CGNS_ENUMT(DofStorage_t) layout;
7  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S, "end");
8  cg_dof_storage_read(&layout);
9
10 /* Read the field data. In a generic restart loop, call
11  * cg_field_info(fn,B,Z,S,F,...) first to discover field names
12  * and types before calling cg_field_read. */
13 cg_field_read(fn, B, Z, S, "Pressure",
14              CGNS_ENUMV(RealDouble), range_min, range_max,
15              pressure_buf);
16
17 /* Reconstruct based on the DOF storage layout */
18 if (layout == CGNS_ENUMV(Shared)) {
19     /* DOFs are shared at mesh vertices/edges/faces.
20      * Map directly to the global DOF vector. */
21     reconstruct_shared(pressure_buf, loc, mesh);
22 } else if (layout == CGNS_ENUMV(Independent)) {
23     /* DOFs are element-local. Each element owns its
24      * own copy of values at shared interfaces. */
25     reconstruct_independent(pressure_buf, loc, mesh);
26 } else {
27     /* DofStorageNull: legacy file. Apply the Section 6.2
28      * inference rules, which for CellCenter and
29      * InterpolationPoints give Independent, and for Vertex
30      * with DataSize == VertexSize give Shared. */
31     reconstruct_legacy(pressure_buf, loc, mesh);
32 }
33
34

```

7.11 Example 11: FSI Coupling — Structural and Fluid Solutions 916

In Fluid-Structure Interaction (FSI) applications, a single zone may carry both fluid 917
and structural solution fields with different DOF storage layouts. A continuous 918
FEM structural solver stores shared displacement DOFs, while a DG fluid solver 919
stores element-local flow variables. Separate FlowSolution_t nodes with distinct 920
DofStorage_t values allow coupling frameworks to correctly interpret and transfer 921
each field. 922

```

1  int F;
2
3  /* Structural solution: shared FEM displacement DOFs */
4  int S_struct;
5  cg_sol_write(fn, B, Z, "StructuralDisplacement",
6              CGNS_ENUMV(Vertex), &S_struct);
7  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S_struct, "end");
8  cg_dof_storage_write(CGNS_ENUMV(Shared));
9  cg_field_write(fn, B, Z, S_struct, CGNS_ENUMV(RealDouble),
10               "DisplacementX", disp_x, &F);
11 cg_field_write(fn, B, Z, S_struct, CGNS_ENUMV(RealDouble),

```

```

12     "DisplacementY", disp_y, &F);
13
14  /* Fluid solution: element-local DG flow field */
15  int S_fluid;
16  cg_sol_write(fn, B, Z, "FluidSolution",
17              CGNS_ENUMV(InterpolationPoints), &S_fluid);
18  cg_sol_interpolation_order_write(fn, B, Z, S_fluid, 3, 0);
19  cg_goto(fn, B, "Zone_t", Z, "FlowSolution_t", S_fluid, "end");
20  cg_dof_storage_write(CGNS_ENUMV(Independent));
21  cg_field_write(fn, B, Z, S_fluid, CGNS_ENUMV(RealDouble),
22               "Pressure", pressure, &F);
23  cg_field_write(fn, B, Z, S_fluid, CGNS_ENUMV(RealDouble),
24               "VelocityX", vel_x, &F);
25
26  /* A coupling framework reading this zone can query each
27   * FlowSolution_t's DofStorage to determine the correct
28   * interpolation and transfer strategy:
29   *   - Shared:      global shared-DOF interpolation
30   *   - Independent: element-local projection */

```

The same pattern applies when the structural and fluid domains are in separate zones. The DOF storage metadata on each zone's `FlowSolution_t` nodes enables the coupling framework to apply the correct inter-code transfer operators without relying on naming conventions or external configuration.

7.12 Example 12: Conjugate Heat Transfer — Perfect and Imperfect Thermal Contact

Conjugate heat transfer (CHT) simulations couple a fluid temperature field (often stored at cell centers or DG DOFs) with a solid temperature field (often stored as a shared-DOF CG FEM solution). The DOF storage metadata disambiguates the two layouts and informs the coupling framework how to perform the interface transfer.

```

1  char basename[CGIO_MAX_NAME_LENGTH+1];
2  int cellDim, physDim, F;
3  cg_base_read(fn, B, basename, &cellDim, &physDim);
4
5  /* ---- Fluid zone: FV/DG temperature (independent) ---- */
6  int S_fluid;
7  cg_sol_write(fn, B, fluidZone, "ThermalSolution",
8              CGNS_ENUMV(CellCenter), &S_fluid);
9  cg_goto(fn, B, "Zone_t", fluidZone,
10         "FlowSolution_t", S_fluid, "end");
11  cg_dof_storage_write(CGNS_ENUMV(Independent));
12  cg_field_write(fn, B, fluidZone, S_fluid,
13               CGNS_ENUMV(RealDouble), "Temperature", T_fluid, &F);
14
15  /* ---- Solid zone: CG FEM temperature (shared) ---- */
16  int S_solid;
17  cg_sol_write(fn, B, solidZone, "ThermalSolution",
18              CGNS_ENUMV(Vertex), &S_solid);
19  cg_goto(fn, B, "Zone_t", solidZone,
20         "FlowSolution_t", S_solid, "end");
21  cg_dof_storage_write(CGNS_ENUMV(Shared));

```

```

22 cg_field_write(fn, B, solidZone, S_solid,
23   CGNS_ENUMV(RealDouble), "Temperature", T_solid, &F);
24
25 /* ---- Interface heat flux on the solid side (face-local) ---- */
26 int SR;
27 cg_subreg_bcname_write(fn, B, solidZone, "FluidInterface",
28   cellDim - 1, "FluidInterface_BC", &SR);
29 cg_goto(fn, B, "Zone_t", solidZone,
30   "ZoneSubRegion_t", SR, "end");
31 cg_dof_storage_write(CGNS_ENUMV(Independent));
32 cg_array_write("HeatFlux", CGNS_ENUMV(RealDouble),
33   1, &nInterfaceFaces, heat_flux);

```

With *perfect* thermal contact the fluid-side and solid-side interface temperatures are equal; the coupling framework can enforce this constraint knowing the solid temperature is **Shared** (shared vertex DOFs) and the fluid temperature is **Independent** (cell-centered).

With *imperfect* thermal contact (finite thermal resistance), a temperature jump ΔT exists at the interface. Each domain retains its own interface temperature, and the coupling framework applies the resistance relation $q = \Delta T/R$ between them. No change to the `DofStorage_t` labels is needed: the fluid and solid temperatures are stored and tagged independently in their respective zones, and the jump is an emergent result of the coupling rather than a property of either field's storage layout.

8 SIDS Impact

1. **New enumeration type:** `DofStorage_t` is added to the SIDS Building-Block Structure Definitions (SIDS Section 4). That section is ordered alphabetically—4.1 `DataClass_t`, 4.2 `Descriptor_t`, 4.3 `DimensionalUnits_t`, 4.4 `DimensionalExponents_t`, 4.5 `GridLocation_t`, 4.6 `IndexArray_t`, 4.7 `IndexRange_t`, 4.8 `Rind_t`—so `DofStorage_t` is inserted as the new **Section 4.5**, and the current 4.5–4.8 are renumbered 4.6–4.9. All cross-references to those four sections require updating.
2. **FlowSolution_t amendment:** The `FlowSolution_t` node definition (SIDS Section 7.7) is amended to include an optional `DofStorage_t` child node.
3. **DiscreteData_t amendment:** The `DiscreteData_t` node definition (SIDS Section 12.4) is similarly amended.
4. **ZoneSubRegion_t amendment:** The `ZoneSubRegion_t` node definition (SIDS Section 7.9) is similarly amended.
5. **UserDefinedData_t amendment:** The `UserDefinedData_t` node definition (SIDS Section 12.10) is similarly amended.

6. **PointList semantics amendment:** The description of **PointList** under **FlowSolution_t** in SIDS §7.7 is amended to permit repeated vertex indices when **DofStorage_t** = **Independent** and **GridLocation_t** = **Vertex**, in unstructured zones only. Repeated indices in that context represent element-local DOFs and are not malformed duplicates. The DOF-to-element mapping, the element traversal order, the treatment of **MIXED** type codes, the exclusion of **NFACE_n**, the whole-zone coverage requirement, and the prohibition on combining **Rind_t** with this encoding are specified in Sections 3.8.1 and 3.8.3 and shall be reproduced in the SIDS text. This amendment does not affect **PointList** semantics under any other condition.
7. **InterpolationPoints interaction:** If CPEX 0045 is adopted, its **GridLocation_t** = **InterpolationPoints** description is amended with the **DofStorage_t** compatibility rules of Section 4.2.
8. No existing SIDS node types, enumerations, or conventions are modified or removed. The only change to existing text is the Section 4 renumbering in item 1, which is editorial.

9 File Mapping Impact

The **DofStorage** child node is stored identically to **GridLocation**: as a character node containing the string name of the enumeration value. The node attributes are given below in the format prescribed by the CGNS File Mapping Manual’s *Detailed CGNS Node Descriptions* [2]. **DofStorage_t** belongs in the “Location and Position Group” of basic nodes, immediately following **GridLocation_t**.

Node Attributes: DofStorage_t

Name:	DofStorage
Label:	DofStorage_t
DataType:	C1
Dimension:	1
Dimension Values:	Length of the string value
Data:	Null, UserDefined, Shared, or Independent
Children:	None
Cardinality:	0,1

This is attribute-for-attribute the same as the **GridLocation_t** node description, apart from the name, label, and permitted data values.

9.1 Affected File Mapping Figures

The following File Mapping figures gain a **DofStorage** child box, drawn in the same style as the existing **GridLocation** child:

- `FlowSolution_t` 987
- `DiscreteData_t` 988
- `ZoneSubRegion_t` 989
- `UserDefinedData_t` 990

9.2 Writing Conventions 991

The node is written only when the DOF storage layout is not `DofStorageNull`; `DofStorageNull` is represented by the absence of the node. This mirrors `GridLocation`, which is likewise omitted when it would carry its default. The two differ in what the absence *means*: an absent `GridLocation` implies the concrete value `Vertex`, whereas an absent `DofStorage` implies only “unspecified” (Section 6.2). 992-997

When `DofStorageUserDefined` is written, the accompanying description node is an ordinary `Descriptor_t` and follows the existing `Descriptor_t` file mapping unchanged, with name `DofStorageDescription`, label `Descriptor_t`, data type `C1`, dimension 1, and cardinality 0,1 within the parent node. 998-1001

10 Design Decisions 1002

Several design questions were evaluated during the development of this proposal. Each is addressed below with rationale grounded in existing CGNS conventions and SIDS precedent. 1003-1005

1. **Scope: `BCData_t` is excluded.** `DofStorage_t` is *not* added to `BCData_t`. Boundary condition data arrays in `BCData_t` are sized by the associated `PointList/PointRange` and `GridLocation_t`, which already determine the data layout unambiguously. Adding `DofStorage_t` to `BCData_t` would therefore be redundant, and would introduce a potential inconsistency if the BC layout differed from the solution it constrains. 1006-1011

No inference in the other direction is intended or valid: the layout of boundary data does *not* follow from the DOF storage layout of the parent `FlowSolution_t`, and weak imposition of boundary conditions is not a DG-specific technique. Nitsche and penalty methods in continuous FEM and in isogeometric analysis routinely pair a `Shared` interior solution with element-local boundary data at face quadrature points. Such data is already expressible: CPEX 0047 permits `GridLocation_t = IntegrationPoint` under `BC_t` and `BCDataSet_t`, with `ItgPointsStartOffset` fixing the per-face layout, so no additional meta-data is required for that case either. The `ZoneSubRegion_t` extension introduced here partially addresses such cases: interface fluxes or surface-local fields that 1012-1021

require a DOF storage label can be stored in a `ZoneSubRegion_t` referencing the relevant boundary condition (see Example 7.12).

2. **Per-solution-node granularity (not per-field).** The DOF storage layout applies at the `FlowSolution_t` level, meaning all `DataArray_t` children share the same layout. This follows the precedent set by `GridLocation_t`, which is likewise a per-solution-node property: all fields within a `FlowSolution_t` node share the same grid location. Per-`DataArray_t` granularity was rejected because it would add significant API and metadata complexity with no demonstrated need. Solvers that genuinely mix layouts within a single zone (e.g., HDG methods with shared traces and independent volumes) should use separate `FlowSolution_t` nodes, exactly as they would use separate nodes for fields at different `GridLocation_t` values.
3. **Semantically inconsistent combinations are permitted but not enforced.** The MLL does *not* enforce consistency between `DofStorage_t` and `GridLocation_t`. Two combinations are defective, in different degrees:
 - A `CellCenter` solution is inherently element-local, so `Shared` is *meaningless* there—the single per-element DOF has no geometric support on the element boundary. `cgnscheck` should warn.
 - `Shared` on an `InterpolationPoints` block whose element set is the zone’s volume elements is *invalid*, not merely odd: the sizing contradicts the definition of `Shared`, and, independently, CPEX 0045 supplies no DOF-to-sub-entity map with which sharing could be realized (Section 4.2). `cgnscheck` should report an error.

In neither case shall the library reject the combination on write. This follows the established CGNS convention of being permissive in what is accepted: the standard documents correct usage; validation is delegated to application-level tools such as `cgnscheck` (Section 11).

4. **Position-based API rather than per-parent functions.** See Section 3.4 for the full rationale. The decisive consideration is that the address-helper pattern used by every other optional scalar metadata child already solves parent dispatch, modify-mode overwrite, and absent-node reads, so reimplementing those three behaviors six times would add code without adding capability.
5. **A new node type rather than a new `GridLocation_t` value.** See Section 3.3.2.

10.1 Rejected Alternatives

A boolean `Discontinuous` flag. Rejected. A boolean cannot express “unspecified,” which is exactly the state every pre-existing file is in, so a boolean would force

writers to assert a layout they may not know. It also leaves no room for a `UserDefined` escape hatch, and CGNS has no precedent for boolean-valued metadata nodes.

A third `MixedStorage` value. Rejected; see the Rationale for Two-Value Classification. Mixed-layout methods store distinguishable data and should store it in distinguishable nodes.

Encoding the layout in `UserDefinedData_t`. Rejected as the primary mechanism. It requires every reader to agree on a name and string convention by side-channel, which is precisely the interoperability failure this CPEX exists to fix. A standard enumeration with a standard label is machine-checkable; `UserDefinedData_t` is not. Note that this CPEX nevertheless *permits* `DofStorage_t` as a child of `UserDefinedData_t`, so applications storing fields there can label them properly.

Deriving the layout from array sizes. Not rejected, and worth stating accurately: for every encoding legal before this CPEX the layout *is* determined by `GridLocation_t` and the SIDS `DataSize` formula, irrespective of element mix (Section 6.2). A discriminator becomes necessary only once a second sizing is admitted at the same grid location, which is what the `Vertex` plus `PointList` encoding of Section 3.8 does. Outside that case the node is confirmatory, and is recommended for uniformity and self-description rather than required to resolve an ambiguity.

Placing the node on `Zone_t` or `Base_t`. Rejected. A single zone routinely carries solutions with different layouts (Examples 7.4, 7.11), so a zone-level setting could not express them.

11 Recommended `cgnscheck` Validation Rules

Per Design Decision 3, the MLL is permissive and validation belongs in `cgnscheck`. The following rules are recommended. Note that the on-disk representation is a C1 string, so there is no such thing as an out-of-range integer value in a file; the corresponding check is on the string.

Error: `DofStorage` node data is not one of "Null", "UserDefined", "Shared", or "Independent". Note that the MLL itself reports this as an error via `cgi_DofStorage` for files at or below the library's own version, and as a warning (degrading to `UserDefined`) for files written by a newer library.

Error: `DofStorage_t` = `Independent` with `GridLocation_t` = `Vertex` and *no* `PointList` — the SIDS `DataSize` formula yields `VertexSize` (the shared-vertex count), which cannot represent the larger element-local DOF count (Section 3.8).

-
- Error:** `DofStorage_t = Independent` with `GridLocation_t = Vertex` and a `PointList`, in a *structured* zone — the ordering rule is defined only for unstructured zones (Section 3.8.1).
- Error:** `DofStorage_t = Independent` with `GridLocation_t = Vertex` and a `PointList` whose contents do not equal the zone's element connectivity traversed per Section 3.8.3. Because the required contents are fully determined, this is an exact check, not a heuristic. It subsumes checks for wrong length, wrong ordering, and incomplete element coverage.
- Error:** `DofStorage_t = Independent` with `GridLocation_t = Vertex` and a `PointList` in a zone containing an `NFACE_n` section (Section 3.8.1).
- Error:** `Rind_t` present alongside `DofStorage_t = Independent`, `GridLocation_t = Vertex`, and a `PointList` (Section 3.8.1).
- Error:** `DofStorage_t = Shared` with `GridLocation_t = InterpolationPoints` and no point set, or with a point set that lists volume rather than interface element indices (Section 4.2).
- Error:** `DofStorage_t = Shared` with `GridLocation_t = IntegrationPoint` — integration-point data is necessarily element-local.
- Warning:** `DofStorage_t = Shared` with `GridLocation_t = CellCenter` (or `FaceCenter`, `EdgeCenter`, `[IJK]FaceCenter`) — semantically inconsistent, since such data is inherently one value per entity with no shared-entity ambiguity (Design Decision 3). **Convention:** this combination should not be used.
- Warning:** `DofStorage_t = Shared` on a `FlowSolution_t` whose `CPEX-0045 InterpolationOrders` gives `spatialOrder = 0` — a $p = 0$ basis has no interface DOFs to share.
- Warning:** A value of `DofStorageUserDefined` without a sibling `Descriptor_t` named `DofStorageDescription` — the convention is then unrecoverable by any reader.
- Warning:** A `DofStorage` node whose data is "Null". This is legal but pointless: the absence of the node carries the same meaning, and writing `Null` through the MLL removes the node rather than creating one, so such a node indicates a file written outside the MLL.

12 Reference Implementation, Versioning, and Test Plan

12.1 Deliverables

The reference implementation comprises:

1. `cgnslib.h`: the `DofStorage_t` enumeration, `NofValidDofStorage`, the `DofStorageName` extern declaration, and prototypes for `cg_dof_storage_read`, `cg_dof_storage_write`, and `cg_DofStorageName`. 1130
1131
1132
2. `cgnslib.c`: the `DofStorageName` table and the three public functions. 1133
3. `cgns_header.h`: the `dof_storage` field in `cgns_sol`, `cgns_discrete`, `cgns_subreg`, and `cgns_user_data`; prototypes for `cgi_dof_storage_address` and `cgi_DofStorage`. 1134
1135
1136
4. `cgns_internals.c`: the address helper, the string mapper, and the four parser updates. 1137
1138
5. `cgns_f.F90` / `cgns_f.h`: the Fortran bindings and the `DofStorage_t` parameter values. 1139
1140
6. `cgnscheck.c`: the validation rules of Section 11. 1141
7. `cgnsview`: display of the new node (no code change expected; the generic C1 node display suffices). 1142
1143
8. Documentation updates to the SIDS, File Mapping Manual, and MLL reference per Sections 8 and 9. 1144
1145

12.2 Versioning 1146

The library version constant is incremented, and the value at which `DofStorage_t` first appears shall be recorded in the SIDS text so that implementers can determine support from the file version alone. The forward-compatibility path described in Section 6 depends on this: `cgi_DofStorage` degrades unrecognized strings to `UserDefined` only when `cg->version > CGNSLibVersion`. 1147
1148
1149
1150
1151

12.3 Test Plan 1152

1. **Round trip, all parents.** For each of `FlowSolution_t`, `DiscreteData_t`, `ZoneSubRegion_t`, and `UserDefinedData_t`: write each of `Shared`, `Independent`, and `DofStorageUserDefined`; close; reopen; read back and compare. 1153
1154
1155
1156
2. **Absent node.** Read from a node with no `DofStorage` child and assert `CG_OK` with `DofStorageNull`. Repeat against a file written by a pre-CPEX-0050 library. 1157
1158
3. **Overwrite in `CG_MODE_MODIFY`.** Write `Shared`, close, reopen in modify mode, write `Independent`, close, reopen: assert the value is `Independent` *and* that the parent has exactly one child with label `DofStorage_t`. This is the regression test for duplicate-child creation. 1159
1160
1161
1162

4. **Deletion.** Write `Shared`, then write `DofStorageNull` in modify mode; assert the child node is gone and a subsequent read yields `DofStorageNull`.

1163
1164
5. **Invalid input.** Assert `CG_ERROR` from `cg_dof_storage_write` for out-of-range enum values.

1165
1166
6. **Wrong parent.** Assert `CG_INCORRECT_PATH` when the current position is, e.g., `GridCoordinates_t`.

1167
1168
7. **No current position.** Assert `CG_ERROR` when called without a preceding `cg_goto`.

1169
1170
8. **Forward compatibility.** Hand-construct a file (via `CGIO`) with `DofStorage` data "Trace" and a version greater than the library's; assert the read yields `DofStorageUserDefined` with a warning, not an error. With a version at or below the library's, assert `CG_ERROR`.

1171
1172
1173
1174
9. **Shared-vertex independent layout.** Build the Example 7.7 file; assert `cg_sol_ptset_info` reports $\text{npnts} = n_{\text{elem}} \times n_{\text{vpe}}$, that `cg_field_read` returns all values, and that `cgnscheck` passes. Then perturb the `PointList` ordering and assert `cgnscheck` reports an error.

1175
1176
1177
1178
10. **cgnscheck rules.** One file per rule in Section 11, asserting the expected error or warning.

1179
1180
11. **Fortran.** Mirror tests 1–4 through the Fortran bindings.

1181
12. **Parallel.** Under `cgp_open`, call `cg_dof_storage_write` collectively from all ranks and assert the file is valid and single-valued; confirm on 1, 2, and 4 ranks.

1182
1183
13. **Backward compatibility of existing tests.** The full existing CGNS test suite shall pass unchanged.

1184
1185

13 Summary of API Additions

1186

Symbol	Description
<i>Public (cgnslib.h)</i>	
<code>DofStorage_t</code>	New enumeration type: <code>DofStorageNull</code> , <code>DofStorageUserDefined</code> , <code>Shared</code> , <code>Independent</code> (see Section 3.2.3 on the unprefixed value names).
<code>DofStorageName</code>	String table for the enumeration values.
<code>NofValidDofStorage</code>	Count of valid enumeration values (4).

Symbol	Description
<code>cg_DofStorageName</code>	C: return the string name for a <code>DofStorage_t</code> value (mirrors <code>cg_GridLocationName</code>).
<code>cg_dof_storage_write</code>	C: write the DOF storage layout of the node at the current position.
<code>cg_dof_storage_read</code>	C: read the DOF storage layout of the node at the current position.
<code>cg_dof_storage_write_f</code>	Fortran binding for <code>cg_dof_storage_write</code> .
<code>cg_dof_storage_read_f</code>	Fortran binding for <code>cg_dof_storage_read</code> .
<i>Internal (cgns_header.h)</i>	
<code>cgi_dof_storage_address</code>	Resolve the current position to the parent's <code>dof_storage</code> field; delete an existing child on modify.
<code>cgi_DofStorage</code>	Map a <code>DofStorage</code> string to the enumeration value.
<code>dof_storage</code>	New field in <code>cgns_sol</code> , <code>cgns_discrete</code> , <code>cgns_subreg</code> , <code>cgns_user_data</code> .

Four public functions cover all four parent node types. No per-parent variants and no `cgp_*` variants are required; see Sections 3.4 and 6.

14 References

References

- [1] CGNS Standard Interface Data Structures (SIDS),
https://cgns.org/standard/SIDS/CGNS_SIDS.html
- [2] CGNS File Mapping Manual—Detailed CGNS Node Descriptions,
<https://cgns.org/standard/FMM/nodes.html>
- [3] CGNS Mid-Level Library—Flow Solution,
https://cgns.org/standard/MLL/api/c_api.html
- [4] CPEX 0045, “Polynomial Data and Curved Grid Elements,”
<https://github.com/CGNS/CGNS/issues/577>

-
- [5] CPEX 0047, “Quadrature Rules Definition and Data Storage” (Integration Point Definition), M. Philit and F. Huvelin, <https://cgns.org/current/CPEX.html>
- [6] Hesthaven, J.S. and Warburton, T., *Nodal Discontinuous Galerkin Methods*, Springer, 2008.
- [7] Cockburn, B., Karniadakis, G.E., and Shu, C.-W., eds., *Discontinuous Galerkin Methods: Theory, Computation, and Applications*, Springer, 2000.
- [8] Miller, M.C., “On the Distinction Between Topological Space and Physical Space in SAF: In other words, what are topo-relations and dof-relations?” ASCI-DMF / SAF Internal Draft, 2003 (rev. 2014).